

Phase 7: Integration & External Access

This phase demonstrates how Salesforce can interact with external systems, simulating a callout to a third-party payment gateway when a booking is confirmed.

Configuration for Callouts

To enable Salesforce to communicate with external endpoints, specific configurations are required.

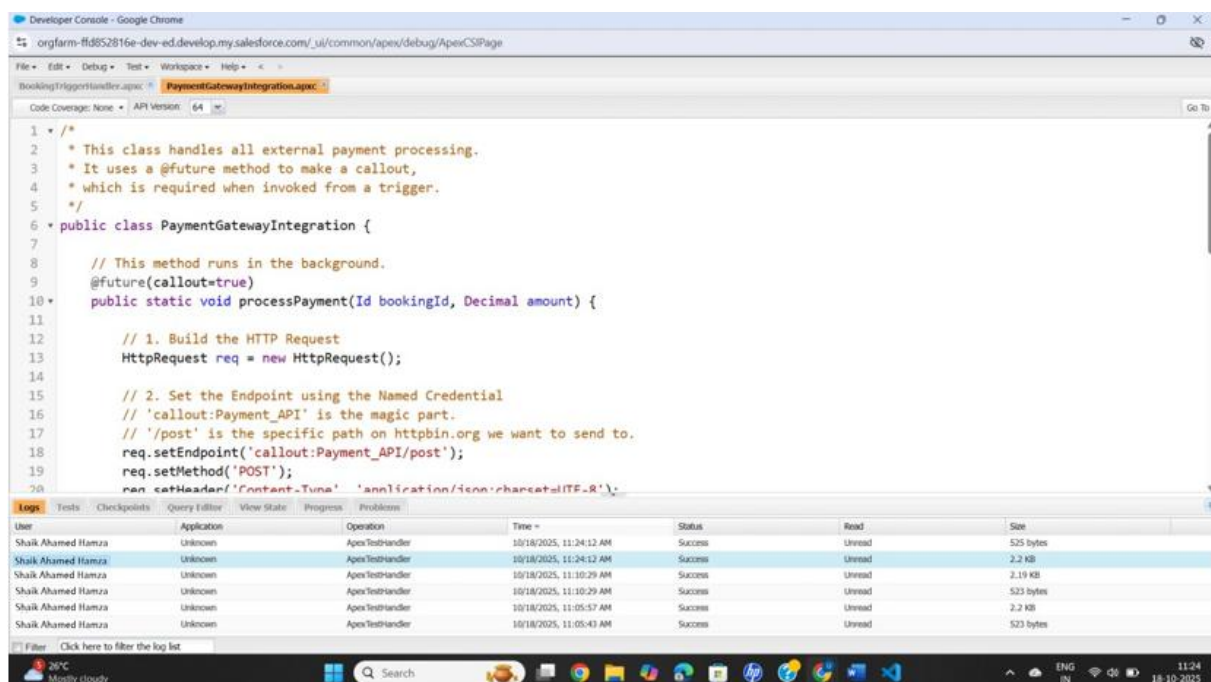
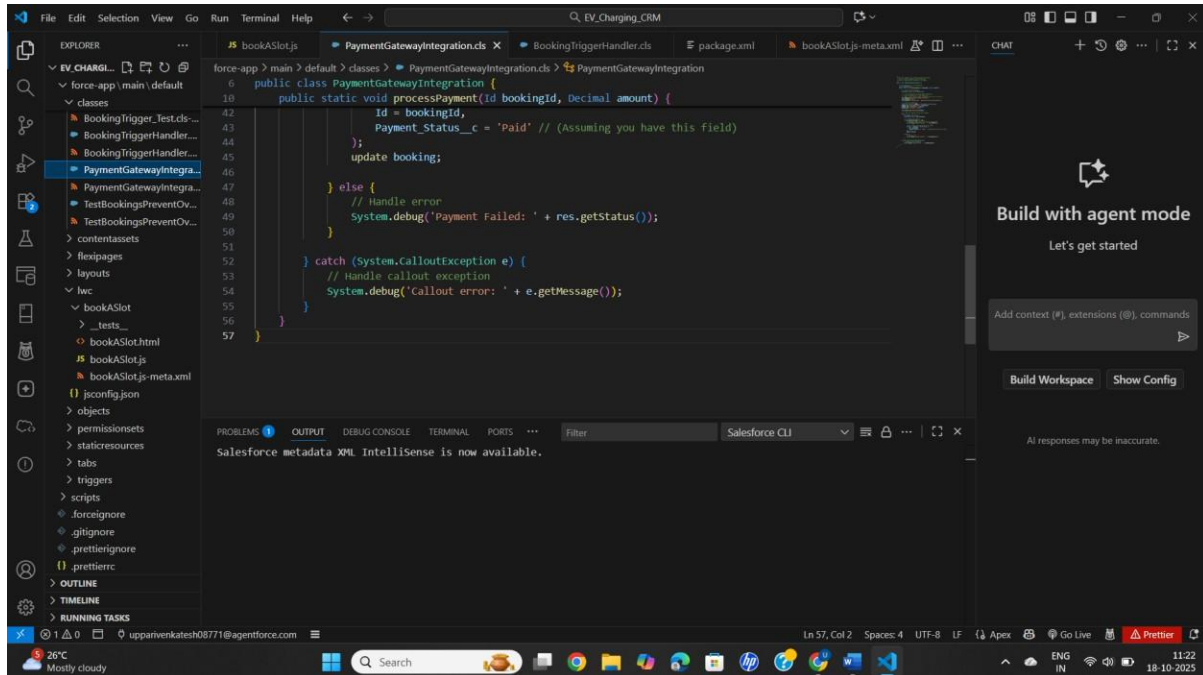
1. Remote Site Settings:

- **Purpose:** The traditional method to whitelist specific domains that Salesforce Apex code is allowed to call out to.
- **Setup:** A Remote Site named Payment_API_Mock was created, allowing callouts to the URL <https://httpbin.org> (a free service for testing HTTP requests).
- **Path:** Setup → Security → Remote Site Settings → New.

2. Named Credentials (Preferred Method):

- **Purpose:** The modern, more secure, and flexible way to define callout endpoints and manage authentication separately from the code.
- **Setup:** A Named Credential named Payment_API was created.
 - **Label:** Payment API
 - **URL:** <https://httpbin.org>
 - **Identity Type:** Anonymous (as httpbin.org requires no login)
 - **Authentication Protocol:** No Authentication
- **Path:** Setup → Security → Named Credentials → New.

- **Usage in Code:** Apex code references this Named Credential using the special URL format `callout:Payment_API/...`, allowing the endpoint URL to be managed declaratively.



Apex Callout Example (Mock Payment Processing)

An Apex class was created to handle the callout asynchronously, as callouts cannot be made directly from triggers.

1. Apex Class: `PaymentGatewayIntegration`

- **Purpose:** Simulates sending booking details to an external payment processor.
- **Method: processPayment(Id bookingId, Decimal amount)**
 - **Annotation:** @future(callout=true) - This allows the method to make external calls and run asynchronously in the background when invoked (e.g., from a trigger).
 - **Logic:**
 1. Constructs an HttpRequest object.
 2. Sets the endpoint using the Named Credential: callout:Payment_API/post. The /post path tells httpbin.org to simply echo back the data sent.
 3. Sets the method to POST.
 4. Sets the Content-Type header to application/json.
 5. Creates a JSON body containing the bookingId and amount.
 6. Uses the Http class to send the request.
 7. Receives the HttpResponse.
 8. Checks the status code (expecting 200 for success from httpbin.org).
 9. Logs the response body using System.debug().
 10. If successful, performs a DML update on the original Booking__c record to set a field like Payment_Status__c to 'Paid'.
 11. Includes **try-catch block** for System.CalloutException to handle potential network or endpoint errors.

2. Trigger Handler Modification:

BookingTriggerHandler.handleAfterUpdate

- **Purpose:** To invoke the payment callout when a booking is confirmed.
- **Logic:** Within the handleAfterUpdate method, a check was added: if a Booking__c record's Status__c changes from any value to 'Confirmed', the PaymentGatewayIntegration.processPayment(bookingId, totalPrice) future method is called, passing the necessary details.

The top screenshot shows the Salesforce IDE with the following code in the `BookingTriggerHandler.cls` file:

```

1 public class BookingTriggerHandler {
2     // (Your handleAfterInsert method is already here...)
3     // public static void handleAfterInsert(List<Booking__c> newBookings) { ... }
4
5     // --- ADD THIS NEW METHOD ---
6     public static void handleAfterUpdate(List<Booking__c> newBookings, Map<Id, Booking__c> oldMap) {
7
8         for (Booking__c newBooking : newBookings) {
9             // Get the old version of the record
10            Booking__c oldBooking = oldMap.get(newBooking.Id);
11
12            // Check if the status "just changed" to 'Confirmed'
13            if (newBooking.Status__c == 'Confirmed' && oldBooking.Status__c != 'Confirmed') {
14
15                // Fire and forget! Call the future method.
16                // We pass the ID and a price.
17                // (You must have a price field on your booking, like Total_Price__c)
18                PaymentGatewayIntegration.processPayment(
19                    newBooking.Id,
20                    newBooking.Total_Price__c // <-- Make sure this field exists!
21                );
22            }
23        }
24    }
25 }

```

The bottom screenshot shows the Salesforce CRM interface for the record `PAY-0001`. The **Details** tab is active, showing the following information:

- Payment Number:** PAY-0001
- Booking:** BK-00012
- Contact:** chakri maddi
- Customer:** Shaik Ahamed Hamza
- Payment Date:** 10/17/2025
- Amount:** \$6,000.00
- Payment Mode:** Success
- Transaction ID:** 25418689526
- Invoice URL:**
- Owner:** Shaik Ahamed Hamza
- Created By:** Shaik Ahamed Hamza
- Last Modified By:** Shaik Ahamed Hamza
- Created Date:** 10/17/2025 3:30 AM
- Last Modified Date:** 10/17/2025 3:30 AM

The **Activity** tab is also visible, showing a section for **Upcoming & Overdue** activities, which currently displays "No activities to show."